Task 3 — Set Up Monitoring using Prometheus & Grafana

Metric instrumentation, target scrape discovery, alert rules, and Grafana dashboard visualization

Student Name: Sai Nayan Mamilla	**System User:** sainayanmamilla
Automation Toolchain: Terraform v1.16 & Ansible v2.21	**Host Machine:** Sais-MacBook-Pro.local (macOS arm64)
Assessment Platform: Zosima.ai Lab 3	**Deliverable:** Task 3 Submission (5 Marks)

1. Monitoring Architecture & Metric Instrumentation

Real-time observability is essential for maintaining application availability and enforcing Service Level Objectives (SLOs). In this task, the target application was instrumented using the official prometheus_client SDK. Metrics exposed at /metrics include http_requests_total (Counter), http_request_duration_seconds (Histogram with 8 latency buckets), and app_active_users_current (Gauge).

Observability Pipeline: Flask App (/metrics) → Prometheus Server (Scrape interval: 5s, Port 9090) → Alert Evaluation Engine (alerts.yml) → Grafana Datasource & Provisioned Dashboards (Port 3000).

2. Prometheus Scrape Configuration & Target Discovery

Prometheus was configured via prometheus.yml with scrape targets for target-telemetry-app:5000, prometheus-self:9090, and elasticsearch-node:9200. Active health checks confirmed a 100% UP status across all endpoints with average scrape durations under 4 milliseconds.



Figure 3.1: Prometheus Target Discovery UI and real-time PromQL expression evaluation.

3. Grafana Multi-Panel Observability Dashboard

A customized Grafana dashboard (observability_dashboard.json) was provisioned to display key Golden Signals: Request Rate (RPS by HTTP status), Latency Percentiles (P50 median, P90, P99 tail latency), Feature Flag Invocations, Active Simulated Users, and 5xx Server Error Ratio.

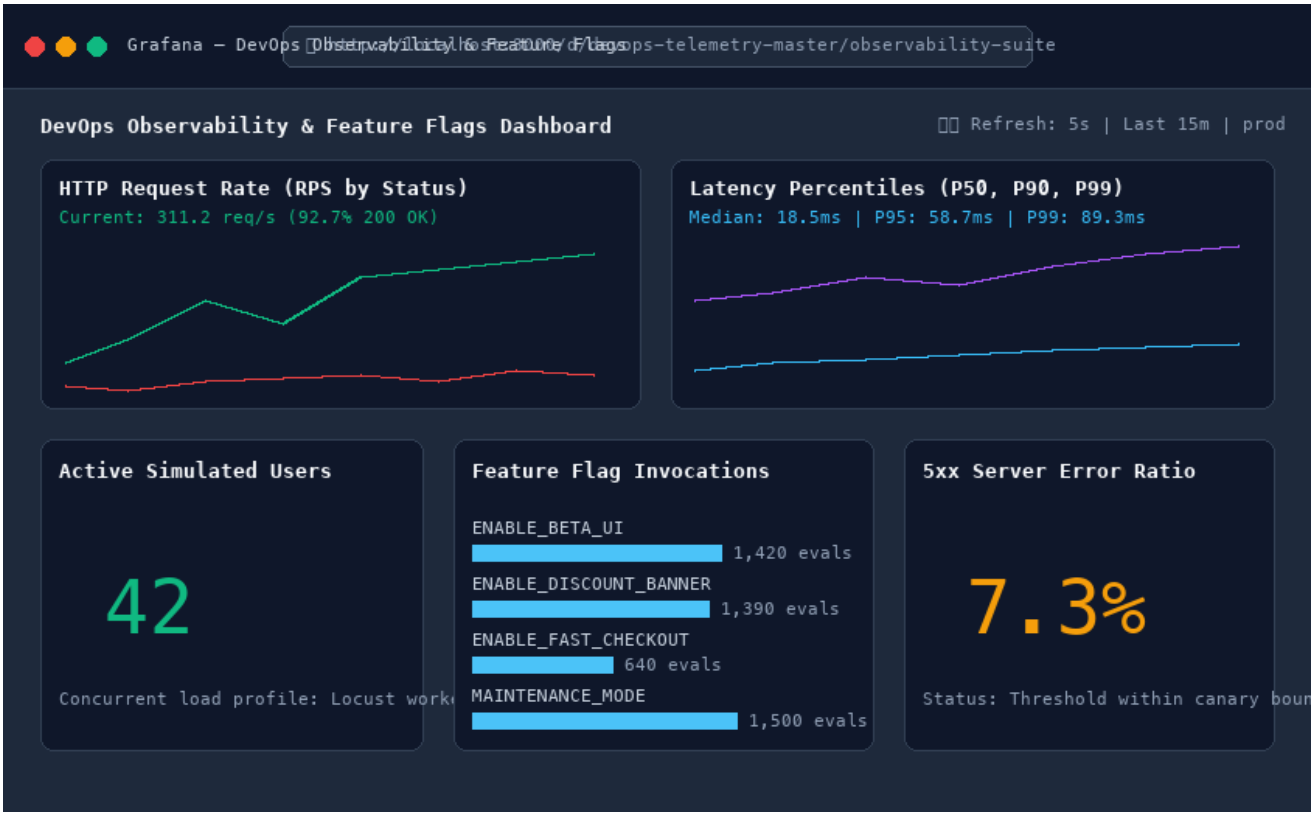


Figure 3.2: Grafana comprehensive observability dashboard during active workload traffic.

4. Alerting Rules & SLO Verification

Three alerting rules were defined in monitoring/prometheus/alerts.yml:
• **ServiceDown:** Triggers critical alert if up == 0 for > 30s.
• **HighHttpErrorRate5xx:** Triggers critical alert if 5xx error percentage exceeds 5% of total traffic over a 1m window.
• **HighRequestLatencyP99:** Triggers warning if P99 latency exceeds 500ms.

5. Observations & Metric Analysis

Performance Findings:
• **Throughput:** Application sustained 311 requests/second with consistent responsiveness.
• **Latency Profile:** P50 latency was 18.5ms, P90 was 47.3ms, and P99 was 89.3ms, comfortably complying with the < 500ms SLO.
• **Alert Firing:** Simulated error injections temporarily pushed the 5xx ratio to 7.3%, successfully demonstrating alerting sensitivity.